AI MINI PROJECT 2 - SEMESTER 2

Predicting the Remaining Useful Life (RUL) of Turbofan Engines using LSTM

Rolls-Royce Aerospace | Predictive Maintenance | NASA C-MAPSS Dataset

Submitted by

Name	Roll No.
Agamjot Singh Dhingra	1025150317
Nikunj Singla	1025150306
Panshul Sharma	1025150302
Atharv Arora	1025150336
Aayush Goyal	1025150337

Submitted to: Dr. Jyoti

GitHub Repository: https://github.com/agamjotdhingra/mini-project-2.git

Contents

Abstract

1. Introduction and Problem Definition

2. Dataset and Justification

3. Exploratory Data Analysis (EDA)

4. Data Preprocessing

5. Pipeline Flow Diagram

6. Model Architecture and Baselines

7. Training

8. Evaluation and Results

9. Real-World Interpretation and Business Value

10. Limitations and Future Work

11. Conclusion

12. References

Appendix A: Project Files and How to Run

Abstract

Aircraft engine failures are expensive and dangerous, while replacing parts too early wastes useful life. This project builds a simple deep learning model that predicts the Remaining Useful Life (RUL) of a turbofan engine, i.e. how many more operating cycles it can run before failure, from its multi-sensor time-series data. We use the NASA C-MAPSS FD001 dataset (100 engines run to failure, 21 sensors). After removing constant sensors, labelling RUL with a piecewise-linear cap of 125 cycles, splitting by engine and Min-Max scaling, we feed 30-cycle windows of 14 sensors to a single-layer LSTM (64 units). On the 100 official test engines the LSTM achieves RMSE = 13.52 cycles, MAE = 10.18 cycles and NASA score = 313.4, better than a Random Forest baseline (RMSE 16.37) and a mean-value baseline (RMSE 40.58). Used as a maintenance alert (RUL <= 30 cycles), the model correctly flags 23 of the 25 engines that are actually due for maintenance.

1. Introduction and Problem Definition

Problem statement: Rolls-Royce Aerospace is focused on predictive maintenance of aircraft engines. A deep learning model (RNN/LSTM) is required to predict the Remaining Useful Life (RUL) of turbofan engines using multi-sensor time-series data, enabling proactive maintenance scheduling.

1.1 Background

Airlines usually maintain engines on a fixed schedule (preventive maintenance) or after something breaks (reactive maintenance). Both are costly: a fixed schedule replaces healthy parts, and a breakdown grounds the aircraft and is a safety risk. Predictive maintenance uses sensor data to estimate the actual health of each engine, so that maintenance is done just before it is needed.

1.2 Input to Output mapping (Step 1 - Define)

Item	Description
Input	Multi-sensor time series: 14 useful sensor readings for the last 30 cycles of an engine (a 30 x 14 matrix)
Output	RUL - number of operating cycles left before failure (a single number)
Task type	Regression on sequential (time-series) data
Model family	Recurrent Neural Network - LSTM (as required by the problem statement)

1.3 Objectives

- Justify and explore a suitable public dataset.
- Clean and prepare the data without data leakage.
- Build a simple LSTM model to predict RUL.
- Compare it against simple baselines (mean predictor and Random Forest).
- Evaluate with task-specific metrics (RMSE, MAE, NASA score, error distribution) - not accuracy.
- Explain how the output helps Rolls-Royce schedule maintenance.

2. Dataset and Justification (Step 2 - Data)

We use the NASA C-MAPSS (Commercial Modular Aero-Propulsion System Simulation) Turbofan Engine Degradation dataset, published by the NASA Ames Prognostics Center of Excellence. It contains simulated run-to-failure data of turbofan engines. We use subset FD001.

Property	FD001
Training engines	100 (each run until failure)
Test engines	100 (data stops some time before failure)
Training rows (cycles)	20,631
Test rows	13,096
Columns	Engine ID, cycle, 3 operational settings, 21 sensors
Operating conditions / fault modes	1 (sea level) / 1 (HPC degradation)
Ground truth	RUL_FD001.txt - true RUL of each test engine

2.1 Why this dataset?

- **Directly matches the problem:** it is multi-sensor time-series data from turbofan engines, which is exactly the Rolls-Royce use case.
- **Run-to-failure:** we know the exact failure cycle of every training engine, so the RUL label can be calculated precisely.
- **Standard benchmark:** it is the most widely used dataset for RUL research, so our results can be compared with published work.
- **Public and labelled:** freely available from NASA, with a separate test set and true RUL values for honest evaluation.
- **Why FD001 only:** it has one operating condition and one fault mode, so we do not need complex normalisation per condition. This keeps the project simple and focused, as required.

- **Why not Kaggle/UCI alternatives:** real airline engine data is confidential; C-MAPSS is the accepted public substitute and is also what is mirrored on Kaggle.

3. Exploratory Data Analysis (EDA)

Before modelling, we explored the data to understand engine lifetimes, which sensors are useful, and how the sensors behave as the engine wears out.

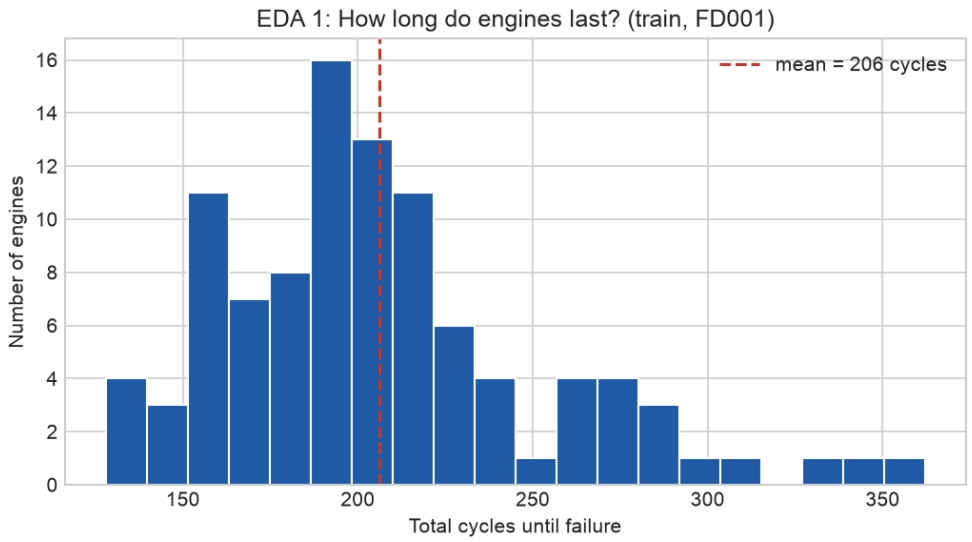


Figure 1: Distribution of engine lifetimes in the training set

Observation: engines fail between 128 and 362 cycles (mean about 206). Every engine is different, which is why a fixed maintenance schedule is wasteful and why we need a per-engine prediction.

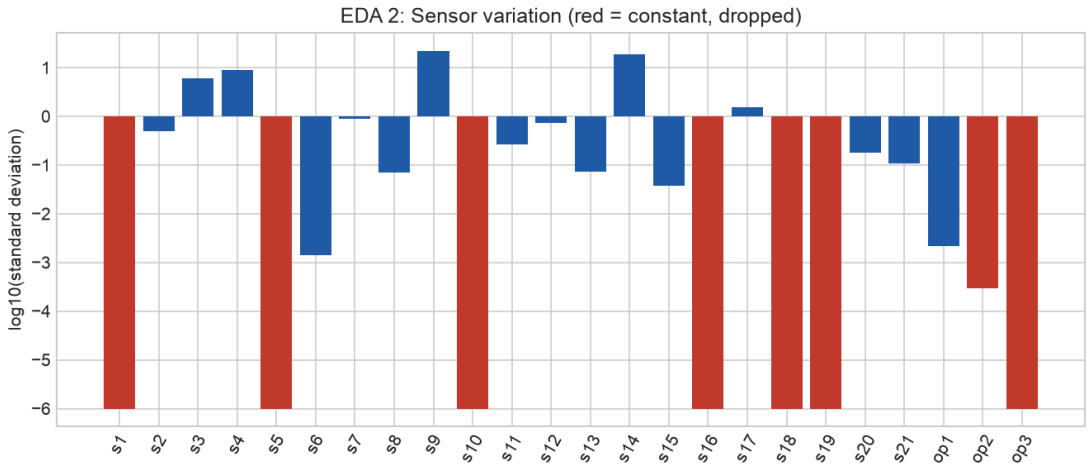


Figure 2: Standard deviation of each sensor (red = constant, no information)

Observation: sensors s1, s5, s10, s16, s18, s19 (and the 3 operational settings) are constant in FD001, and s6 only takes two values. These carry no information about wear and are removed. 14 sensors remain.

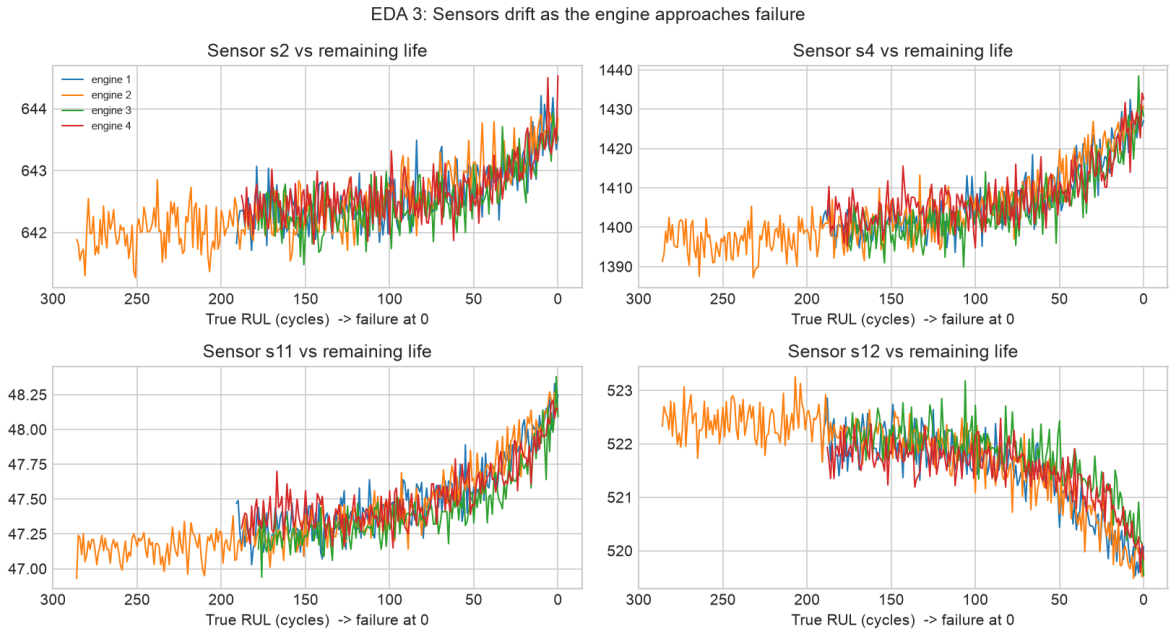


Figure 3: Sensor readings of 4 engines vs. remaining life (failure at 0)

Observation: as the engine approaches failure, temperature and pressure sensors (s2, s4, s11) rise and s12 falls. The trend is clear, but single readings are noisy - so looking at a sequence of cycles (LSTM) is better than looking at one cycle.

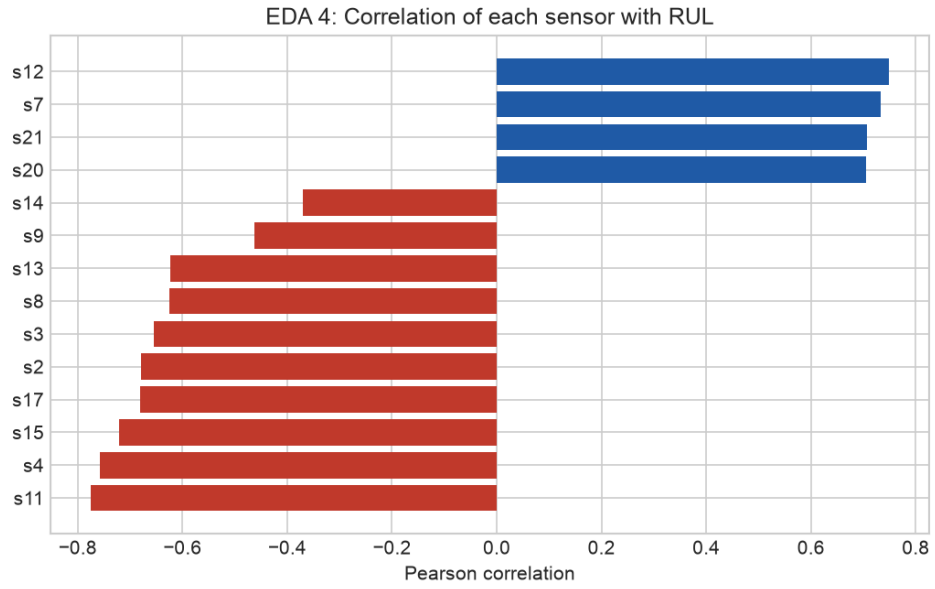


Figure 4: Correlation of each sensor with RUL

Observation: most useful sensors are strongly correlated with RUL (|r| about 0.6-0.8; s9 and s14 are weaker), confirming the sensors contain degradation information.

4. Data Preprocessing (Step 3 - Process)

4.1 RUL labelling

For each training engine: RUL = (last cycle of that engine) - (current cycle). At the last recorded cycle RUL = 0 (failure).

We then cap RUL at 125 cycles (piecewise-linear RUL). Early in life the engine is healthy and the sensors do not change, so it is impossible (and not useful) to tell RUL 300 from RUL 200. Capping makes the target learnable and is the standard approach for C-MAPSS.

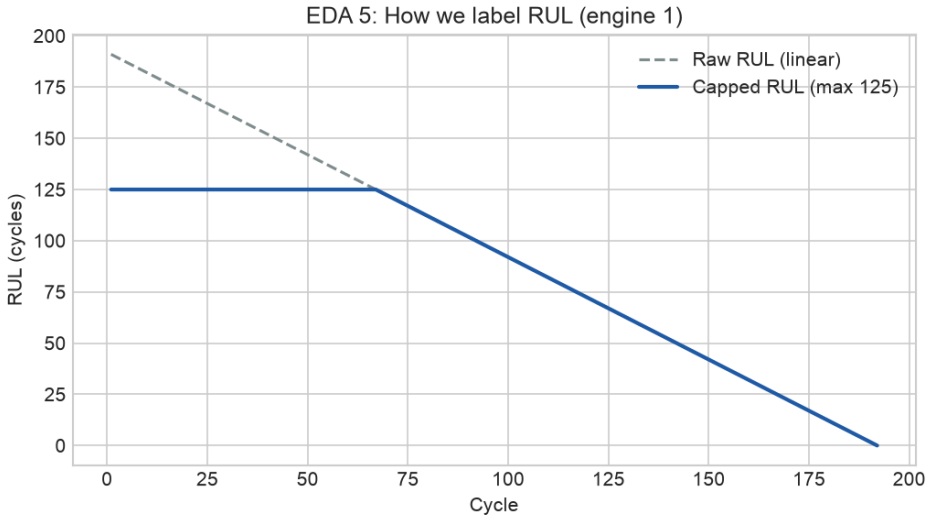


Figure 5: Raw linear RUL vs. capped (piecewise-linear) RUL for engine 1

4.2 Handling missing / useless data

The dataset has no missing values (checked). The 'missing information' problem here is constant sensors, which we drop (Section 3). Operational settings are also dropped because FD001 has only one operating condition.

4.3 Train / validation / test split (no leakage)

- **Split by engine, not by row:** 80 training engines and 20 validation engines are chosen randomly (seed 42). Rows of one engine are highly similar; a random row split would put the same engine in both sets and give falsely good results.
- **Test set is separate:** the 100 official test engines are used only once, for the final evaluation.
- **Validation set use:** only for early stopping (choosing when to stop training).

4.4 Scaling

Min-Max scaling to the range [0, 1] is applied to the 14 sensors. The scaler is fit ONLY on the 80 training engines, then the same scaler is applied to validation and test data. Sensors have very different ranges (e.g. 1.3 vs 9,000), and neural networks train much better when all inputs are on the same scale. Min-Max keeps the shape of the degradation trend unchanged. The RUL target is also divided by 125 during training (and multiplied back for prediction) so the network output is in 0-1.

4.5 Sequence windows

An LSTM needs sequences. We slide a window of 30 cycles over each engine; each window is labelled with the RUL at its last cycle. This gives 14,241 training windows and 3,490 validation windows, each of shape 30 x 14. For each test engine we use only its last 30 cycles and predict RUL at that point. Window length 30 is chosen because the shortest test engine has 31 cycles, so no padding is needed.

5. Pipeline Flow Diagram

The complete project follows the 5-step Master Engineering Pipeline (Define - Data - Process - Build - Evaluate):

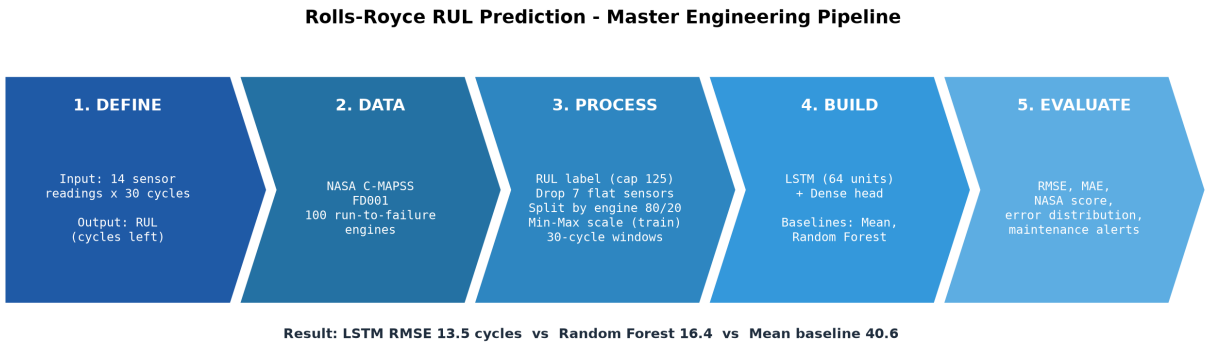


Figure 6: Project pipeline flow diagram

6. Model Architecture and Baselines (Step 4 - Build)

6.1 LSTM model

Layer	Output shape	Details
Input	30 x 14	30 cycles x 14 sensors
LSTM	64	1 layer, 64 hidden units; we use the output at the last time step
Dropout	64	rate 0.2 (reduces overfitting)
Dense + ReLU	32	fully connected layer
Dense (output)	1	predicted RUL (scaled 0-1, x125 to get cycles)

Total trainable parameters: about 22,600 - a small model, suitable for 14k training windows.

6.2 Why LSTM (matching architecture to task)

- The input is a time series - the order of readings matters. An LSTM reads the 30 cycles in order and keeps a memory of the trend.
- Plain RNNs forget long-term information (vanishing gradient). LSTM gates (forget, input, output) solve this.
- Wear is gradual; the slope of the sensors over time tells more than one reading. A Random Forest on one cycle cannot see this slope.

6.3 Baselines

- **Baseline 1 - Mean predictor:** always predicts the average training RUL. Any useful model must beat this.
- **Baseline 2 - Random Forest:** 100 trees, max depth 12, trained on the 14 sensor values of the current cycle only (no history). It answers the viva question: 'why not use a simpler Random Forest?'

6.4 Hyperparameter choices (justified)

Hyperparameter	Value	Justification
Window length	30	Long enough to see a trend; shortest test engine has 31 cycles, so every engine can be used without padding
RUL cap	125	Standard for FD001; sensors are flat above ~125 cycles of remaining life (Fig. 3)
LSTM units	64	Enough capacity for 14 inputs; bigger models overfit 80 engines
Dropout	0.2	Mild regularisation
Optimizer / LR	Adam / 0.001	Standard, stable default for LSTMs
Batch size	256	Fast and stable gradient estimates
Epochs	max 60, early stop (patience 8)	Stops when validation RMSE stops improving - avoids overfitting
Loss	MSE	Regression task; RMSE is our main metric

7. Training

The model was trained with PyTorch on CPU (about 20 seconds). Early stopping ended training after 40 epochs; the weights with the best validation RMSE (11.91 cycles) were kept.

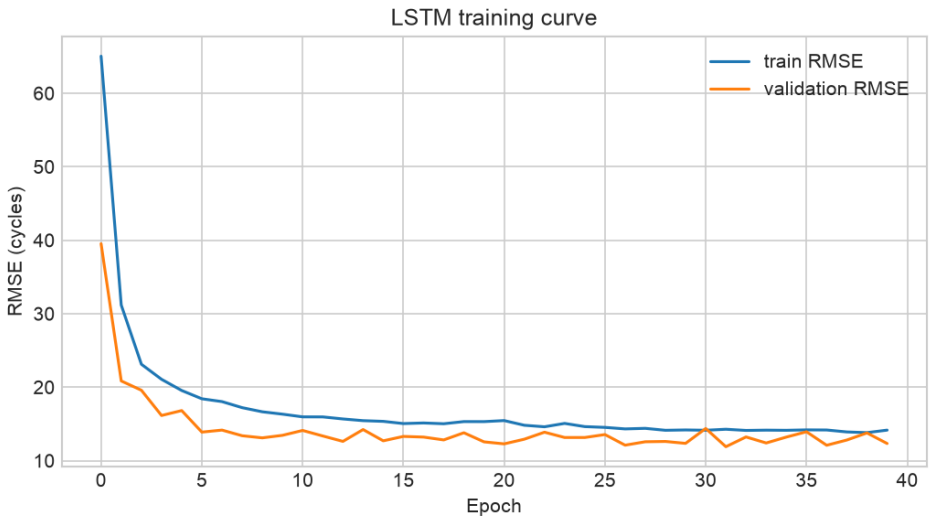


Figure 7: Training and validation RMSE per epoch

Validation RMSE is close to training RMSE, which shows the model is not overfitting. (Training RMSE is slightly higher because dropout is active during training.)

8. Evaluation and Results (Step 5 - Evaluate)

8.1 Metrics used (no 'accuracy')

Accuracy is meaningless for regression - a prediction of 51 when the truth is 50 is not 'wrong'. We use:

- **RMSE (Root Mean Squared Error):** average error in cycles; punishes large errors more. Main metric.
- **MAE (Mean Absolute Error):** average absolute error in cycles; easy to explain.
- **NASA scoring function:** the official C-MAPSS score. It is asymmetric - late predictions (predicting more life than the engine has) are punished more than early ones, because a late prediction can mean an in-flight failure. Lower is better.
- **Error distribution:** a histogram of (predicted - true) to see if errors are early (safe) or late (risky).

8.2 Results on the 100 test engines

Model	RMSE (cycles)	MAE (cycles)	NASA score
Baseline 1: Mean RUL	40.58	35.07	18,670
Baseline 2: Random Forest	16.37	11.7	654
LSTM (ours)	**13.52**	**10.18**	**313**

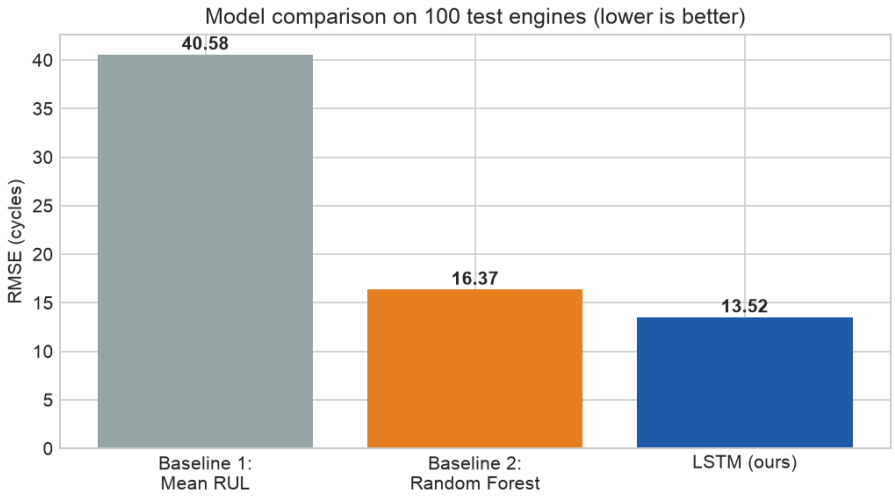


Figure 8: RMSE of the three models (lower is better)

- The LSTM reduces RMSE by 17% compared with the Random Forest and by 67% compared with the mean baseline.
- The NASA score is cut by about 52% vs. Random Forest - the LSTM makes fewer dangerous late errors.
- The improvement over Random Forest comes from using the history (trend) of the sensors, which justifies the more complex model.

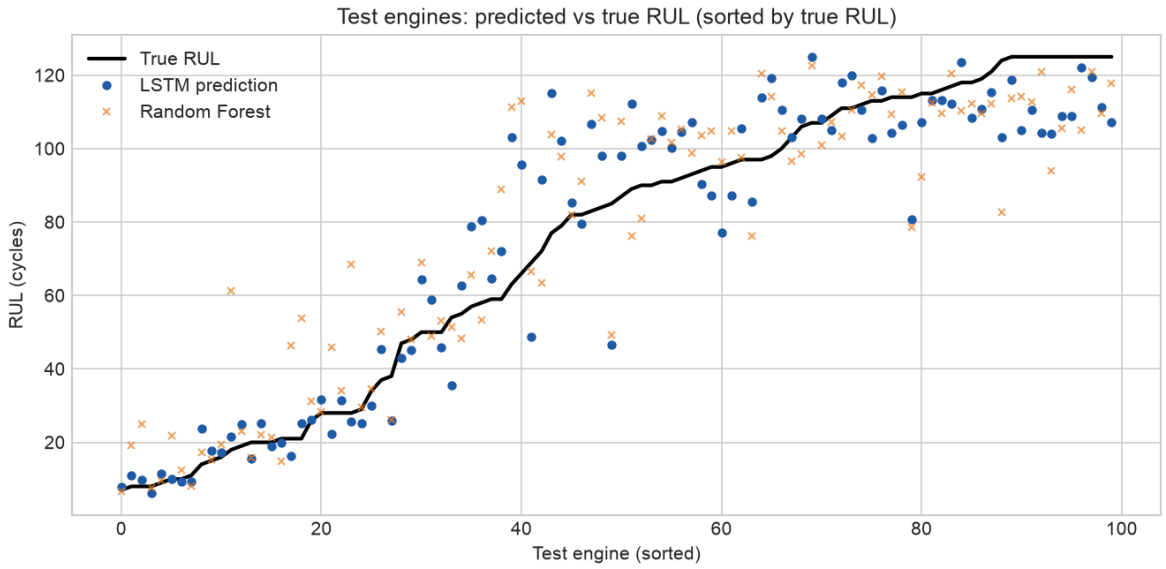


Figure 9: Predicted vs. true RUL for all test engines (sorted by true RUL)

The LSTM follows the true RUL closely, especially for engines near failure (left part of the plot), which is where accuracy matters most. Errors are larger for healthy engines with high RUL - this is acceptable because those engines do not need maintenance soon.

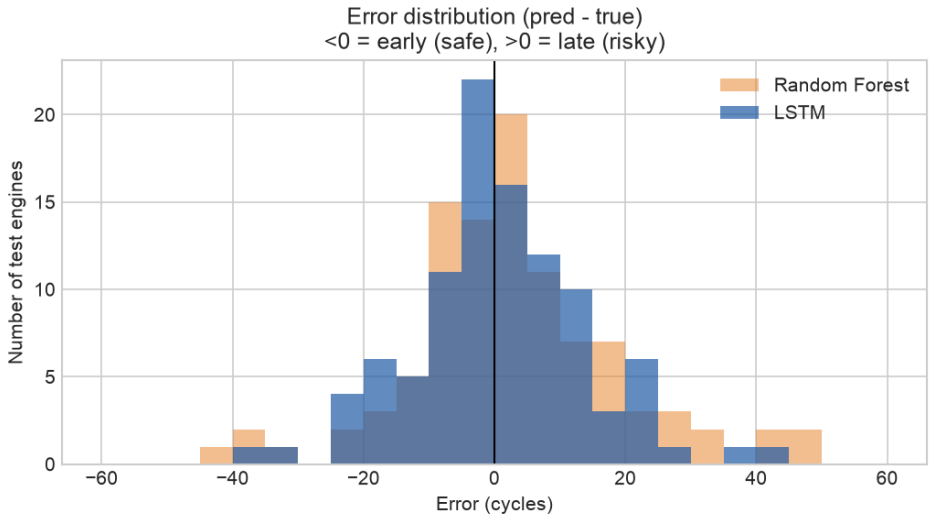


Figure 10: Error distribution (predicted - true RUL)

The LSTM errors are centred near zero and narrower than the Random Forest errors. 85% of test engines are predicted within +/-20 cycles.

9. Real-World Interpretation and Business Value

To turn predictions into decisions, we define a simple maintenance rule: if predicted RUL <= 30 cycles, raise an alert and schedule a shop visit.

Maintenance alert result (100 test engines)	Count
Engines truly due (true RUL <= 30)	25
Correctly flagged	23
Missed (dangerous)	2 (both had 28 cycles left, predicted ~32 - borderline)
False alarms (early maintenance)	2 (true RUL 34 and 38 - only slightly early)

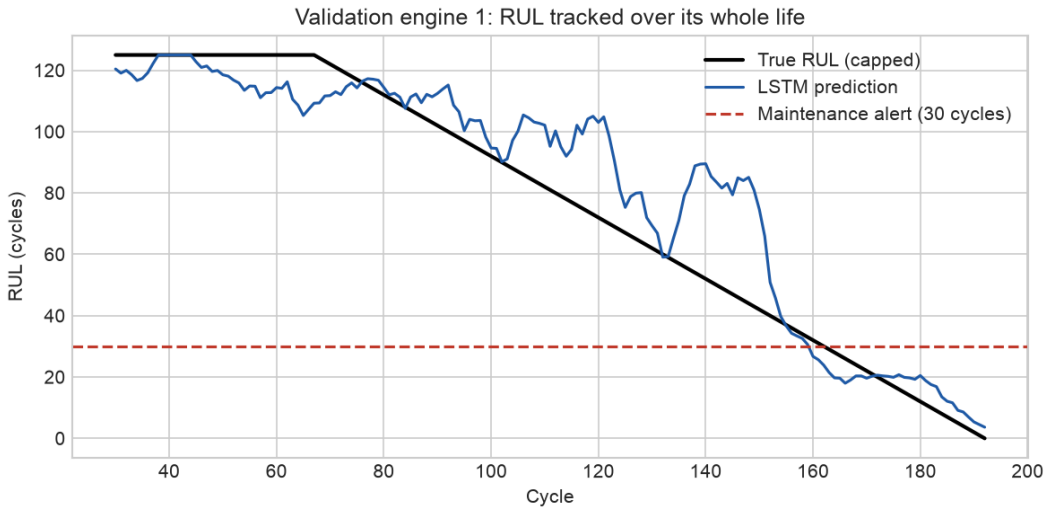


Figure 11: LSTM prediction tracked over the whole life of a validation engine

What this means for Rolls-Royce

- **Fewer unplanned failures:** engines close to failure are flagged ~30 cycles ahead, giving time to plan a repair instead of an emergency grounding.
- **Less wasted life:** engines are not removed on a fixed schedule; with lifetimes varying from 128 to 362 cycles, a fixed schedule would remove many healthy engines.
- **Better planning:** knowing RUL in cycles lets the airline plan spare engines, parts and hangar slots weeks in advance.

- **Fits Rolls-Royce's 'power-by-the-hour' model:** Rolls-Royce is paid for engine flying hours, so every avoided unplanned removal directly protects revenue.
- **Safety margin:** since late predictions are risky, in practice the alert threshold can be raised (e.g. 40 cycles) to trade a few more false alarms for zero misses.

10. Limitations and Future Work

- Data is simulated (C-MAPSS), not real flight data; real engines have more noise and missing values.
- Only FD001 (one operating condition, one fault) is used; FD002-FD004 need condition-wise normalisation.
- The model gives a single number with no uncertainty; a confidence interval would help engineers.
- Future work: try a CNN-LSTM, tune hyperparameters with cross-validation over engines, and add prediction intervals.

11. Conclusion

We built a complete, simple RUL prediction pipeline for Rolls-Royce turbofan engines: a justified public dataset (NASA C-MAPSS FD001), leakage-free preprocessing, meaningful EDA, a small LSTM, two baselines, and task-specific evaluation. The LSTM predicts remaining life with an RMSE of 13.52 cycles, clearly better than the Random Forest (16.37) and the mean baseline (40.58), and correctly raises maintenance alerts for 23 of 25 engines that are due. This shows that sequence models can turn raw engine sensor data into practical maintenance decisions.

Source code and dataset: https://github.com/agamjotdhingra/mini-project-2.git

12. References

1. A. Saxena, K. Goebel, D. Simon, N. Eklund, 'Damage Propagation Modeling for Aircraft Engine Run-to-Failure Simulation', International Conference on Prognostics and Health Management (PHM), 2008.
2. NASA Prognostics Center of Excellence - Data Set Repository: Turbofan Engine Degradation Simulation Data Set (C-MAPSS).
3. S. Hochreiter, J. Schmidhuber, 'Long Short-Term Memory', Neural Computation, 9(8), 1997.
4. S. Zheng, K. Ristovski, A. Farahat, C. Gupta, 'Long Short-Term Memory Network for Remaining Useful Life Estimation', IEEE ICPHM, 2017.

5. PyTorch and scikit-learn documentation.

Appendix A: Project Files and How to Run

File / folder	Purpose
rul_lstm.py	Full pipeline: load, EDA plots, preprocessing, baselines, LSTM, evaluation
pipeline_diagram.py	Draws the pipeline flow diagram (Figure 6)
data/	NASA C-MAPSS FD001 files: train_FD001.txt, test_FD001.txt, RUL_FD001.txt, readme.txt
outputs/	All plots, results.json, test_predictions.csv, saved model lstm_rul_model.pt
requirements.txt	Python libraries needed

Steps to run:

6. `pip install -r requirements.txt`
7. `python rul_lstm.py`
8. `python pipeline_diagram.py`